

Дисциплина: Математическая логика и теория алгоритмов (Лекция 1)

ВУЗ: Казанский федеральный университет (КФУ)

Группа / Курс: Для студентов 2 курса бакалавриата

Специальность: 09.03.02 Информационные системы и технологии

Профиль: Разработка DevSecOps

Дата лекции: 16 сентября 2026 года

Исходный видеофайл: мат логика 16.09.mp4

Текст транскрипции: [lecture-01-transcript.txt](#)



Оглавление

1. [Введение. Информационные процессы и формализация](#)
 2. [Булева алгебра и логические операции](#)
 3. [Оптимизация и техническая реализация](#)
 4. [Комбинационные схемы и автоматы с памятью](#)
 5. [Представление булевых функций и Совершенные формы](#)
 6. [Базисы ИЛИ-НЕ и И-НЕ](#)
 7. [Синтез комбинационных схем: Сумматор](#)
 8. [Будущее программирования и абстракции](#)
-

1. Введение. Информационные процессы и формализация [00:00:00 - 00:30:00]

Лекция начинается с обсуждения важности математической логики в контексте современных вычислительных систем. - **Информационные процессы:** Развитие от первых вычислительных машин к распределённым системам (интернету) и до современных прорывов в области искусственного интеллекта. - **Цель курса:** Освоить математический аппарат (в первую очередь, символическую или формальную логику) для формального описания информационных процессов. - **Синтаксис против семантики:** Формальные системы опираются исключительно на синтаксис (правила

манипулирования символами), а не на семантику (смысл). Это критически важно для автоматизации, чтобы компьютер мог выполнять преобразования без участия человека.

- **Цифровой логический уровень:** Самый нижний уровень архитектуры компьютера описывается на языке булевой алгебры (с некоторыми расширениями).

2. Булева алгебра и логические операции [00:40:00 - 01:00:00]

Джордж Буль изначально разрабатывал свой математический аппарат для формализации законов мышления, однако этот язык оказался идеальным для описания работы переключательных схем.

Базовые операции (Функционально полная система)

В классической булевой алгебре выделяют три ключевые операции: 1. **Инверсия (Логическое отрицание, НЕ):** Превращает 0 в 1, а 1 в 0. В схемотехнике реализуется инвертором. 2. **Конъюнкция (Логическое умножение, И):** Истинна только тогда, когда истинны оба операнда. 3. **Дизъюнкция (Логическое сложение, ИЛИ):** Истинна, если истинен хотя бы один операнд.

 **ВНИМАНИЕ: Ошибка обозначения** Лектор строго критикует использование знака плюса (+) для обозначения дизъюнкции, так как логическое сложение не тождественно арифметическому.

Импликация (Следование)

- Формула $A \rightarrow B$ (из A следует B).
- **Антицедент** (A) — основание, **Консеквент** (B) — следствие.
- Импликация ложна **только в одном случае:** когда антицедент истинен, а консеквент ложен ($1 \rightarrow 0 = 0$). Во всех остальных случаях она истинна.

3. Оптимизация и техническая реализация [01:10:00 - 01:20:00]

- **Математическая избыточность:** В математике стоит задача выразить закон меньшим количеством знаков.
- **Техническая реализация:** С технической точки зрения меньшее количество символов часто означает более простую схему. Однако это не всегда так: иногда

более длинная формула на бумаге требует меньшего числа транзисторов при физической реализации.

- **Равносильность (Эквивалентность):** Две формулы A и B равносильны, если на одинаковых наборах входных значений они принимают одинаковые истинностные значения ($A \equiv B$).

4. Комбинационные схемы и автоматы с памятью [01:20:00 - 01:30:00]

- **Комбинационные схемы:** Результат (выходной сигнал) формируется за один интервал времени исключительно на основе текущей комбинации входных сигналов.
- **Автоматы (Последовательностные схемы):** Вводится понятие **времени** и **состояния**.
- Преобразования выполняются шаг за шагом (уровень микроархитектуры).
- Состояние схемы зависит не только от текущих входов, но и от предыдущего состояния автомата.

5. Представление булевых функций и Совершенные формы [01:50:00 - 02:00:00]

Функции можно задавать таблицами истинности или аналитически. - **Совершенная**

дизъюнктивная нормальная форма (СДНФ): - Строится по строкам таблицы

истинности, где функция равна 1. - Позволяет сразу видеть, на скольких и на каких

именно наборах функция принимает единичное значение. - **Геометрическое**

представление: Функцию от n переменных можно представить в виде n -мерного куба (например, для 3 переменных — трехмерный куб, где вершины — это наборы значений).

6. Базисы ИЛИ-НЕ и И-НЕ [02:10:00 - 02:20:00]

- **Стрелка Пирса (ИЛИ-НЕ, NOR):** Инверсия дизъюнкции **Math input error**.
- **Штрих Шеффера (И-НЕ, NAND):** Инверсия конъюнкции $y = \overline{x_1 \wedge x_2}$.
- Обе эти функции образуют функционально полные системы (базисы), позволяя выразить любую булеву функцию только через один тип вентиляей.
- **Влияние элементной базы:** Переход на другой тип транзисторов (например, полевые вместо биполярных) на физическом уровне привел к архитектурному

сдвигу от централизованных ЭВМ к распределенным системам.

7. Синтез комбинационных схем: Сумматор [02:20:00 - 02:40:00]

- **От логики к арифметике:** Построение арифметических операций на базе логических элементов.
- **Полусумматор (Half-Adder):** Складывает два бита.
- **Полный сумматор (Full-Adder):** Складывает два бита с учетом бита переноса из предыдущего разряда.
- Сегодня этапы синтеза и оптимизации сильно автоматизированы (САПР). Проектировщики используют высокоуровневые языки описания аппаратуры (HDL), а транслятор переводит их в логические вентили.

8. Будущее программирования и абстракции [02:40:00 - 02:50:00]

- Разработка смещается на все более высокие уровни абстракции. Программист оперирует готовыми блоками (как сумматор), не задумываясь о внутренней реализации на вентилях.
- С развитием нейросетей (LLM) ручное написание кода на языках высокого уровня уходит в прошлое. Компьютер (компилятор/нейросеть) лучше справляется с оптимизацией и трансляцией намерений в машинный или промежуточный код.